

ISS projekt 2025/26 – Music Search

Tomáš Janoušek

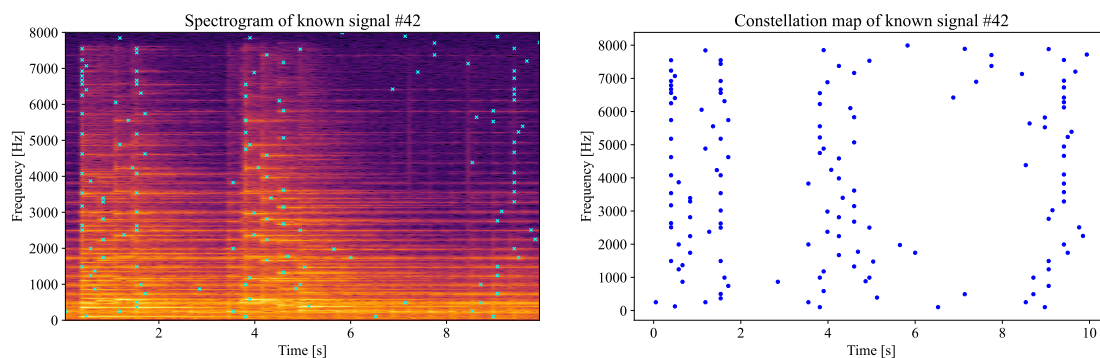
Login: 239026

22. listopadu 2025

Úvod

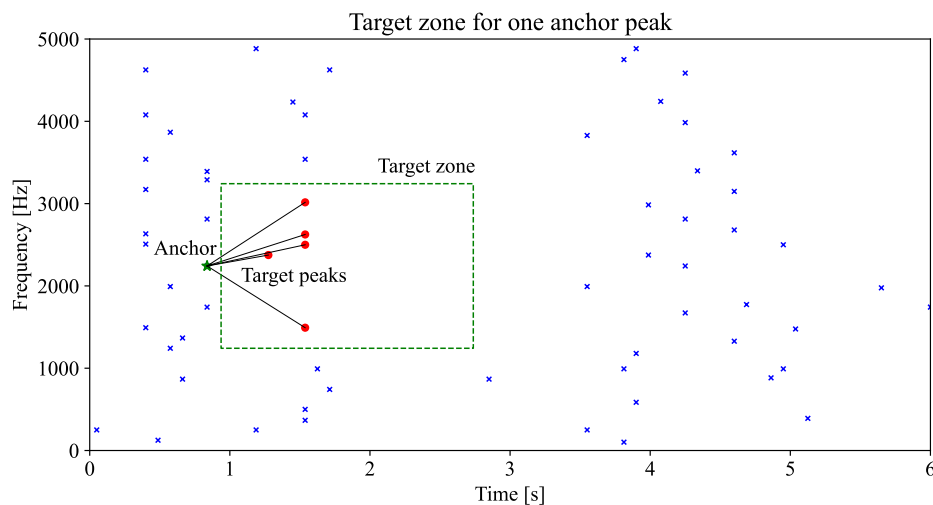
Cílem tohoto projektu je vytvořit program, který dokáže přiřadit zašuměné nebo jinak znehodnocené úryvky hudby k odpovídajícím skladbám ve známé databázi. Ačkoliv existuje velmi mnoho přístupů k řešení tohoto problému, z osobního zájmu jsem se rozhodl zkusit implementovat zjednodušený algoritmus služby Shazam. Důvodem pro toto rozhodnutí byla skutečnost, že jsem službu Shazam dlouhé roky využíval jako uživatel a vždy mě zajímalo (a fascinovalo), jakým způsobem je možné z krátké a často velmi nekvalitní nahrávky spolehlivě určit konkrétní skladbu. Implementace vlastní verze tohoto přístupu mi umožnila detailně porozumět jeho principu a ověřit si jeho robustnost v praxi.

Základním zdrojem, ze kterého se v tomto řešení vychází je Shazam whitepaper - *An Industrial-Strength Audio Search Algorithm* [1], který představuje algoritmus, který je robustní a efektivní. Dále bylo vycházeno z různých videí a internetových článků popisujících tento algoritmus - např. [2], [3], [4]. Základní myšlenkou Shazam algoritmu je vytvoření fingerprintu – kompaktní reprezentace zvukového signálu, která zůstává stabilní i při výrazném zhoršení kvality nahrávky. Tento fingerprint vzniká analýzou spektrogramu zvukové nahrávky, ze kterého se extrahují pouze nejvýraznější lokální maxima, tzv. „spectrogram peaks“. Ty tvoří tzv. konstelační mapu viz obr. 1, tedy řídkou množinu bodů v časově-frekvenčním prostoru. Tyto body představují robustní zvukové události, které obvykle „přežijí“ i agresivní degradaci signálu [1].



Obrázek 1: Vlevo: spektrogram v logaritmické škále s vyznačenými lokálními maximy. Vpravo: konstelační mapa z nejsilnějších spektrálních peaků.

Jakmile je vytvořena konstelační mapa, následuje klíčová fáze, v níž se z jednotlivých peaků (bodů v konstelační mapě) generují tzv. „hashe“. Jde o postup, který výrazně zvyšuje entropii fingerprintu a umožňuje velmi rychlé a přesné vyhledávání ve velké databázi. Důležitou vlastností hashe v kontextu tohoto algoritmu je, že je časově invariantní - jeho hodnota nezávisí na absolutní poloze v signálu (to bude důležité zejména pro přiřazování neznámého signálu ke známému). Při hashování je pro každý bod v konstelační mapě definována tzv. cílová zóna (target zone), která zahrnuje body ve vymezeném časovém a frekvenčním rozsahu. Bod pro který je daná cílová zóna vytvořena se nazývá kotva (anchor point) viz obr. 2. Všechny body nacházející se v jeho cílové zóně jsou s ním následně propojeny pomocí hashů.



Obrázek 2: Příklad cílové zóny.

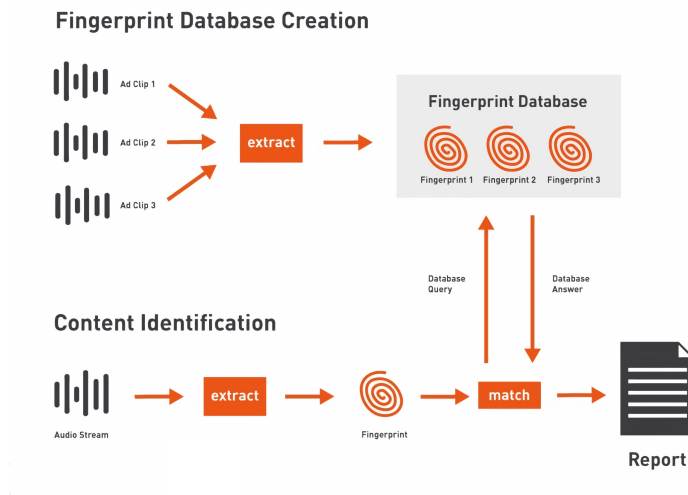
Každý hash musí obsahovat:

- f_1 - frekvenci kotvy (anchor pointu)
- f_2 - frekvenci daného bodu v cílové zóně
- $\Delta t = t_2 - t_1$ - časový rozdíl mezi těmito dvěma body

Tento postup vytvoření fingerprintu (spektrogram $\rightarrow$ konstelační mapa $\rightarrow$ hash) je poté aplikován na všechny známé zvukové signály. Výstupem fingerprintu každé skladby je množina hashů, které jsou následně uloženy spolu s dalšími informacemi (identifikátor skladby, čas výskytu daného hashe ve skladbě) do databáze.

Pro identifikaci neznámého signálu se výše uvedený postup aplikuje beze změny na neznámý signál. Výstupem je opět sada hashů, které jsou následně využity k nalezení odpovídajících hashů v databázi. Neznámý signál je na závěr identifikován jako skladba v databázi, se kterou má nejvíce shodných hashů s odpovídajícími časovými offsety. Detailnější popis výše uvedeného procesu je k dispozici v [1]. Zjednodušený princip algoritmu je možné vidět na obr. 3.

Wang dále v [1] zmiňuje, že je tento přístup mimořádně odolný – spolehlivou identifikaci skladby lze dosáhnout i tehdy, když přežije pouhé 1–2 % původních hashů. Díky tomu lze rozpoznávat skladby i v extrémně hlučném prostředí, za přítomnosti jiného hudebního zdroje nebo po kompresi formátem GSM. Právě tato kombinace robustnosti, rychlosti a přesnosti dělá ze Shazam algoritmu jeden z významných příspěvků v oblasti zpracování audio signálu.



Obrázek 3: Zjednodušený princip Shazam algoritmu [5].

Implementace

Prvním krokem implementace bylo vytvoření funkce `#extract_peaks` pro získání reprezentativní množiny vrcholů ze spektrogramu analyzovaného signálu, viz alg. 1. Pro získání samotného spektrogramu byla využita funkce z knihovny SciPy [6]. Spektrální píky odpovídají lokálním maximům v časově-frekvenčním prostoru a tvoří základ již zmiňované konstelační mapy. Pro jejich detekci byl využit postup popsáný v [1], založený na aplikaci lokálního maximálního filtru. Tento filtr porovnává hodnotu každého bodu spektrogramu s maximem v jeho okolí definovaném pevnou velikostí okna. Body, které odpovídají skutečným lokálním maximům, jsou následně extrahovány. Všechny detekované peaky jsou poté seřazeny podle magnitudy a ponecháno je 256 peaků s nejvyšší magnitudou, viz algoritmus 1. Takto získaná množina bodů tvoří vstup pro následnou konstrukci hashů.

Algorithm 1: Extraction of peaks from the spectrogram

Input: Frequency vector f , time vector t , spectrogram S , neighborhood size N , max peaks K

Output: Peak indices `peaksIdx`, peak positions `peaks`

$\text{localMax} \leftarrow \text{MAXIMUMFILTER}(S, N)$

$\text{peakMask} \leftarrow (S == \text{localMax})$

$(\text{freqIdx}, \text{timeIdx}) \leftarrow \text{WHERE}(\text{peakMask})$

$\text{magnitudes} \leftarrow S[\text{freqIdx}, \text{timeIdx}]$

$\text{sortedIdx} \leftarrow \text{SORTDESCENDING}(\text{magnitudes})$

$\text{nPeaks} \leftarrow \text{length}(\text{sortedIdx})$

if $\text{nPeaks} > K$ **then**

$\text{sortedIdx} \leftarrow \text{sortedIdx}[0:K]$

$\text{nPeaks} \leftarrow K$

end

$\text{freqIdx} \leftarrow \text{freqIdx}[\text{sortedIdx}], \text{timeIdx} \leftarrow \text{timeIdx}[\text{sortedIdx}]$

$\text{peaksIdx} \leftarrow \text{ZEROS}(\text{nPeaks}, 2), \text{peaks} \leftarrow \text{ZEROS}(\text{nPeaks}, 2)$

$\text{peaksIdx}[:, 0] \leftarrow \text{freqIdx}, \text{peaksIdx}[:, 1] \leftarrow \text{timeIdx}$

$\text{peaks}[:, 0] \leftarrow f[\text{freqIdx}], \text{peaks}[:, 1] \leftarrow t[\text{timeIdx}]$

return $(\text{peaksIdx}, \text{peaks})$

Po extrakci nejvýznamnějších vrcholů spektrogramu následuje další fáze konstrukce fingerprintu, jež byla již zmíněna v úvodu, v níž jsou vytvořeny hashe (**#hash_generation**, viz algoritmus 2). Každý bod v konstelační mapě je nejprve zvolen jako kotva (anchor point), ke které se vyhledají všechny body nacházející se v její cílové zóně (target zone) viz obr. 2. Tato zóna (viz **#target_zone**) je definována pevně zvoleným časově-frekvenčním rozsahem a představuje množinu bodů, se kterými může být kotva spárována. Pro každou takovou dvojici kotvy a cílového bodu je následně vytvořen hash, který obsahuje frekvenci kotvy, frekvenci cílového bodu a časový rozdíl mezi nimi. Tato trojice $(f_1, f_2, \Delta t)$ je poté převedena na jednoznačný hash pomocí hashovací funkce. Jako hashovací funkce byla využita built-in funkce pythonu `hash()`. Pokud je hash generován v režimu registrace zvukového signálu, pak se ukládá spolu s časem výskytu kotvy a identifikátorem skladby; v režimu identifikace se ukládá pouze samotný hash a čas kotvy. Výsledkem je kompaktní a robustní reprezentace zvukového signálu, která se následně používá při vyhledávání shod v databázi.

Algorithm 2: Hash generation from the constellation map

Input: List of peaks P , optional song index s
Output: List of hashes
`numPeaks` $\leftarrow$ `length(P)`
`hashList` $\leftarrow$ empty list
for $i \leftarrow 0$ **to** `numPeaks` **do**
 `anchor` $\leftarrow P[i]$
 $(\text{targetPeaks}, _)$ $\leftarrow$ `TARGETZONESEARCH(anchor, P)`
 if *targetPeaks is empty* **then**
 continue
 end
 foreach $\text{target} \in \text{targetPeaks}$ **do**
 `freq1` $\leftarrow$ `int(anchor.freq)`
 `freq2` $\leftarrow$ `int(target.freq)`
 `t1` $\leftarrow$ `int(anchor.time × 100)`
 `t2` $\leftarrow$ `int(target.time × 100)`
 `deltaT` $\leftarrow$ `t2 − t1`
 `hashVal` $\leftarrow$ `HASH((freq1, freq2, deltaT))`
 if s *is provided* **then**
 append (`hashVal`, `t1`, s) to `hashList`
 end
 else
 append (`hashVal`, `t1`) to `hashList`
 end
 end
end
return `hashList`

Funkce **#fingerprint_generation** pro generování fingerprintu propojuje všechny uvedené kroky do jednoho celku, viz algoritmus 3. Ze vstupního audio signálu je nejprve vypočten spektrogram. Z něj jsou následně extrahovány peaky tvořící konstelační mapu a tyto píky jsou poté využity pro tvorbu hashů, které představují finální fingerprint použitelný jak pro registraci, tak i pro identifikaci skladby.

Algorithm 3: Fingerprint generation pipeline

Input: Audio signal x , optional song index s
Output: Fingerprint hash list
 $(f, t, S) \leftarrow \text{SPECTROGRAM}(x)$
 $(_, \text{peaks}) \leftarrow \text{EXTRACTPEAKS}(f, t, S)$
 $\text{hashes} \leftarrow \text{GENERATEHASHES}(\text{peaks}, s)$
return hashes

Pro registraci všech známých signálů byla vytvořena funkce `#build_database`, která zajišťuje vytvoření databáze hashů všech 706 referenčních skladeb. Pro každý známý signál je nejprve vygenerován jeho fingerprint a všechny vytvořené hashe jsou následně uloženy do slovníkové struktury. Tento přístup je plně postačující pro demonstrační účely a menší datové sady. Pro větší datové sady by však bylo mnohem vhodnější využít skutečný databázový systém, například SQLite, PostgreSQL apod.

Aby nebylo nutné při každém spuštění skriptu vytvářet databázi znovu, byly vytvořeny pomocné funkce `#save_database` a `#load_database`, které umožňují databázi uložit a načíst ze souboru.

Pro identifikaci neznámého zvukového signálu je na závěr použita funkce `#recognize_signal`, která porovnává fingerprint vstupního signálu s již vytvořenou databází hashů. Nejprve je pro vstupní nahrávku vygenerován fingerprint stejným způsobem jako při registraci známých skladeb, viz. obrázek 3. Následně jsou všechny hashe z této sady postupně porovnány s položkami v databázi a u každého nalezeného shodného hashe je inkrementován počet odpovídajících výskytů dané skladby. Výsledkem je mapa počtů shod pro jednotlivé skladby, ze které lze určit, se kterou známou nahrávkou má vstupní signál největší podobnost. Tento postup popsán pseudokódem 4 představuje jednoduchý, ale účinný způsob identifikace založený na počtu shodných hashů.

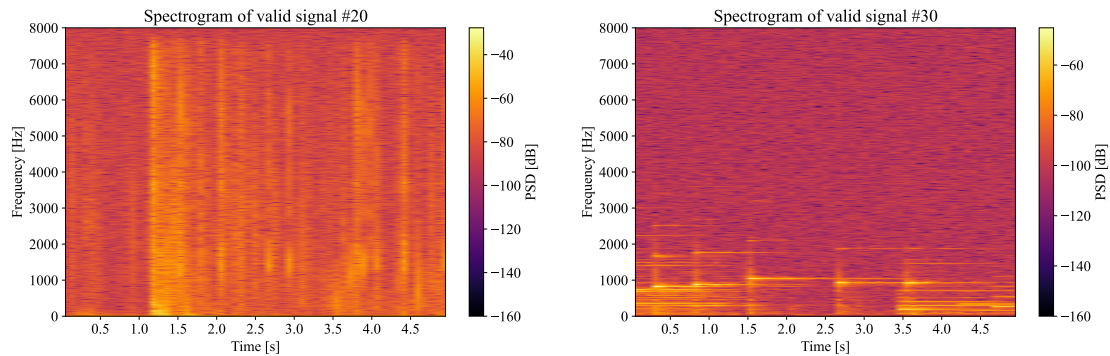
Funkci `#comp_similarity_matrix` lze využít pro výpočet počtu shod pro větší množinu signálů. Tato funkce postupně iteruje přes všechny vstupní signály, které mají být identifikovány a pro každý z nich zavolá rozpoznávací funkci `#recognize_signal`. Získané počty shodných hashů jsou následně uloženy do jediné matice.

Algorithm 4: Recognition of an unknown signal

Input: Input audio signal x , fingerprint database DB
Output: Match counts per song
 $\text{queryHashes} \leftarrow \text{GENERATEFINGERPRINT}(x)$
 $\text{matches} \leftarrow \text{empty map (song} \rightarrow \text{count)}$
foreach $(\text{hashVal}, t) \in \text{queryHashes}$ **do**
 if $\text{hashVal} \in DB$ **then**
 $\text{entries} \leftarrow DB[\text{hashVal}]$
 foreach $\text{songIdx} \in \text{entries}$ **do**
 $\text{matches}[\text{songIdx}] \leftarrow \text{matches}[\text{songIdx}] + 1$
 end
 end
end
return matches

Výsledky a zhodnocení

Pro ověření funkčnosti implementovaného systému byla provedena série experimentů na validační sadě o celkovém počtu 50 zvukových signálů. Každý signál byl identifikován na základě porovnání jeho fingerprintu s databází známých skladeb. Program dosáhl celkové úspěšnosti 98 %, což potvrzuje, že zvolený postup je velmi účinný i v případě relativně jednoduché implementace. Většina nahrávek byla identifikována jednoznačně již na základě počtu shodných hashů. V podstatě lze říci, že jediným problematickým signálem byl signál *valid #20*, který po manuálním poslechu obsahuje pouze šum. Jedná se pravděpodobně o začátek nebo konec některé ze známých desetisekundových nahrávek a je zcela opodstatněné, že jej algoritmus nedokázal identifikovat, protože se jedná o velmi obtížně identifikovatelný úsek. Podrobnější analýza spektrogramu problematického signálu *valid #20* (viz obr. 4 vlevo) to potvrzuje. Spektrogram obsahuje pouze rovnoměrně rozprostřenou energii napříč celým frekvenčním spektrem bez přítomnosti jakýchkoliv stabilních harmonických frekvencí. Jedná se tedy pravděpodobně o zmíněný šum. Pro porovnání byl přiložen i obrázek jednoho z dobře identifikovatelných signálů *valid #30*, viz obrázek 4 vpravo.



Obrázek 4: Spektrogramy validačních signálů #20 a #30.

Významnou, avšak méně viditelnou část projektu tvořilo ladění jednotlivých parametrů, které mají zásadní vliv na kvalitu výsledného fingerprintu i na přesnost identifikace. Největší pozornost byla věnována nastavení cílové zóny, zejména jejímu časovému posunu a délce, protože tyto parametry určují, které píky jsou vzájemně párovány při tvorbě hashů. Stejně důležitá byla i volba parametrů spektrogramu, především velikost okna.

Ladění probíhalo rovněž v oblasti extrakce peaků. Byly prozkoumány různé přístupy extrakce vrcholů, přičemž některé nebyly příliš úspěšné - například přístup založený na prahování. V tomto případě byla vytvořena maska lokálních maxim spektrogramu, která byla následně prořezána na základě prahové hodnoty odvozené od maximální hodnoty ve spektrogramu. Cílem bylo odstranit méně významné vrcholy a zachovat pouze dominantní akustické události. Tento postup se však ukázal jako málo robustní - vhodná volba prahu byla silně závislá na konkrétním signálu. V některých případech docházelo k úplnému vymizení klíčových vrcholů, což následně znemožňovalo vytvoření konzistentního fingerprintu. Z těchto důvodů byl tento způsob extrakce opuštěn a nahrazen robustnějším přístupem založeným na detekci lokálních maxim. V této variantě je spektrogram analyzován pomocí funkce *maximum_filter* z balíčku *scipy.ndimage* [6], která pro každý bod vyhledá maximální hodnotu v jeho okolí definovaném pevným oknem (parametr *neighborhood_size*). Vrcholy jsou následně určeny jako body, jejichž hodnota

přesně odpovídá lokálnímu maximu v daném okně. Tento postup se projevil jako výrazně stabilnější než prahování.

Možná zlepšení

Ačkoli dosažené výsledky ukazují, že implementované řešení je schopné velmi přesně identifikovat krátké zvukové signály, existuje několik oblastí, ve kterých by bylo možné algoritmus dále vylepšit. Jedním z nich je způsob vyhodnocování shody mezi fingerprintem neznámého signálu a databází známých skladeb.

V tomto projektu je rozhodnutí založeno pouze na počtu shodných hashů, zatímco originální algoritmus Shazamu [1] navíc ještě porovnává i časové offsety jednotlivých hashů. Tento přístup umožňuje určit navíc i čas v originální nahrávce (uložené v DB), ze kterého neznámý úsek pochází. Důvodem proč nebylo v tomto projektu nutné uvažovat časové offsety hashů je ten, že mají všechny skupiny signálů pevně danou délku (známé nahrávky 10 s, validační a testovací 5 s). Ve skutečných podmínkách by však takto zjednodušené vyhodnocování velmi pravděpodobně nestačilo. Různé skladby mají různé délky a jejich struktura se může výrazně lišit. Navíc databáze často obsahují nejen hudební skladby, ale také delší řečové nahrávky či podcasty. Tyto typy obsahu přirozeně budou obsahovat podstatně větší množství hashů, což vede k tomu, že i zcela nesouvisející úsek může vykazovat velký počet náhodných shod s dlouhými signály v databázi. Z tohoto důvodu je v praxi nezbytné zohlednit i časové offsety, které umožňují identifikovat skutečné shluky odpovídajících hashů a tím odlišit reálné shody od náhodných, pro více informací viz [1].

Dodatek

Dodatečně byla vytvořena i jednoduchá experimentální implementace porovnávání signálů `#recognize_signal_offs`, která uvažuje také časové posuny hashů. Tato funkce umožňuje též zjistit čas, ze kterého úsek pochází. Výsledky této funkce nebyly v rámci projektu detailněji analyzovány, ale na validační množině dosahuje stejných výsledků - tedy 98 % úspěšnost identifikace. Výsledná evaluace, jejímž výstupem je soubor eval.txt, byla provedena pomocí lépe otestované funkce `#recognize_signal`.

Reference

- [1] Avery Li-Chun Wang. „An Industrial-Strength Audio Search Algorithm“. In: *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*. 2003, s. 7–13. URL: <http://www.ee.columbia.edu/~dpwe/papers/Wang03-shazam.pdf>.
- [2] Computerphile. *How Shazam Works (Probably!)* <https://www.youtube.com/watch?v=RRsq9apr5QY>. YouTube video, accessed 2025-11-19. 2020.
- [3] Jovan Jovanovic. *How does Shazam work? Music recognition algorithms, fingerprinting, and processing*. <https://www.toptal.com/algorithms/shazam-it-music-processing-fingerprinting-and-recognition>. Toptal article, accessed 2025-11-19. 2018.
- [4] Chigozirim Igweamaka. *SeekTune: An implementation of Shazam’s song recognition algorithm*. <https://github.com/cgzirim/seek-tune>. GitHub repository, accessed 2025-11-19. 2025.
- [5] Edem Gold. *How does all this work in Shazam*. 2022. URL: <https://hackernoon.com/how-shazam-works-in-a-nutshell>.
- [6] Pauli Virtanen et al. „SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python“. In: *Nature Methods* 17 (2020), s. 261–272. DOI: 10.1038/s41592-019-0686-2.